Java(sem)-4

Q1: Explain the steps involved in creating and working with user-defined packages with an example

Definition

A **package** in Java is a namespace used to organize related classes and interfaces into a hierarchical structure. A **user-defined package** is created by the programmer to group their own classes for better organization, reusability, and security.

Syntax

package packageName;

Steps to Create and Use a User-Defined Package (Explanation Only)

• **Step 1:** Create a Java class and declare a package using the package keyword at the top.

• **Step 2:** Compile the program using the -d option to generate the proper folder structure.

• **Step 3:** The compiler creates a directory with the package name and stores the .class file inside it.

• **Step 4:** Create another Java program to use the package.

• **Step 5:** Import the package using the import keyword or use the fully qualified class name.

• **Step 6:** Create an object of the class and call its methods.

Example (Implementation with Code)

File 1: Message.java

```
package mypack;

public class Message {
   public void show() {
      System.out.println("Hello from package");
   }
}
```

Compile Command

```
javac -d . Message.java
```

File 2: TestPackage.java

```
import mypack.Message;

public class TestPackage {
    public static void main(String[] args) {
        Message m = new Message();
        m.show();
    }
}
```

Alternative (Without Import)

```
public class TestPackage {
    public static void main(String[] args) {
        mypack.Message m = new mypack.Message();
        m.show();
    }
}
```

Output

Hello from package

Key Points

• Avoids naming conflicts
• Provides access control
• Improves code organization
• Supports reusability
• Uses package and import keywords

Conclusion

User-defined packages help in organizing Java programs effectively, making them modular, reusable, and easy to maintain.

2. Discuss access control for class members across different packages. Illustrate with examples.

Definition

Access control in Java defines the **visibility and accessibility of class members (variables and methods)**. It is implemented using **access modifiers**: public, protected, default, and private.

Syntax

```
accessModifier dataType variableName;

accessModifier returnType methodName() {
   // code
}
```

Access Modifiers (Detailed Explanation)

• **public**
Members declared as public are accessible **from anywhere**, i.e., within the same class, same package, and even from different packages (with import).

• **protected**
Members declared as protected are accessible within the **same package** and also in **different packages only through inheritance** (subclass). They cannot be accessed directly using objects in non-subclasses outside the package.

• **default (no modifier)**
If no access modifier is specified, it is called **default access**. These members are accessible **only within the same package** and cannot be accessed outside the package.

• **private**
Members declared as private are accessible **only within the same class**. They are not visible in subclasses or other classes, even in the same package.

Summary Table

Modifier	**Same Class**	**Same Package**	**Different Package (Subclass-inheritence)**	**Different Package (Non-Subclass)**
public	YES	YES	YES	YES
protected	YES	YES	YES	NO
default	YES	YES	NO	NO
private	YES	NO	NO	NO

Example (Across Different Packages)

File 1: pack1/A.java

```
package pack1;

public class A {
    public int a = 10;
    protected int b = 20;
    int c = 30;          // default
    private int d = 40;

    public void display() {
        System.out.println(a + " " + b + " " + c + " " + d);
    }
}
```

File 2: pack2/B.java (Subclass in Different Package)

```
package pack2;

import pack1.A;

public class B extends A {
    public static void main(String[] args) {

        B obj = new B();

        System.out.println(obj.a); // public → accessible
        System.out.println(obj.b); // protected → accessible (inheritance)

        // System.out.println(obj.c); // ERROR (default)
        // System.out.println(obj.d); // ERROR (private)
    }
}
```

File 3: pack2/C.java (Non-Subclass)

```
package pack2;

import pack1.A;
```

```
public class C {
    public static void main(String[] args) {

        A obj = new A();

        System.out.println(obj.a); // public → accessible

        // System.out.println(obj.b); // ERROR
        // System.out.println(obj.c); // ERROR
        // System.out.println(obj.d); // ERROR
    }
}
```

Output

```
10
20
10
```

Key Points

• public → accessible everywhere
• protected → accessible outside package only via inheritance
• default → not accessible outside package
• private → accessible only within class
• Provides **security and encapsulation**

Conclusion

Access control in Java helps regulate how class members are accessed across packages. It ensures proper encapsulation, security, and controlled usage of data in object-oriented programming.

3. Java Program to Copy Even Numbers into Even.txt and Odd Numbers into Odd.txt

Definition (Concept Used)

Character Stream (File Stream)
A character stream in Java is used to read and write text data. It handles data in the form of characters. Classes like FileWriter and FileReader are used for this purpose.

FileWriter
FileWriter is a class used to write character data into a file. It automatically creates the file if it does not exist.

Approach

- Create an array of numbers
- Open two files: Even.txt and Odd.txt
- Traverse each number
- If number is even → write to Even.txt
- If number is odd → write to Odd.txt
- Close both files

Syntax

```
FileWriter fw = new FileWriter("fileName.txt");
fw.write(data);
fw.close();
```

Program

```
import java.io.FileWriter;
import java.io.IOException;

public class EvenOddFile {
   public static void main(String[] args) {

      int[] nums = {10, 15, 20, 25, 30, 33};

      try {
         FileWriter evenFile = new FileWriter("Even.txt");
         FileWriter oddFile = new FileWriter("Odd.txt");
```

```
            for (int n : nums) {
                if (n % 2 == 0) {
                    evenFile.write(n + " ");
                } else {
                    oddFile.write(n + " ");
                }
            }

            evenFile.close();
            oddFile.close();

            System.out.println("Data written successfully");
        } catch (IOException e) {
            System.out.println("Error occurred: " + e.getMessage());
        }
    }
}
```

Output

Data written successfully

Even.txt

10 20 30

Odd.txt

15 25 33

Key Points

- FileWriter is used to write character (text) data into files
- It belongs to character stream
- Files are created automatically if not present
- close() is necessary to flush and save data
- IOException should be handled properly

Conclusion

This program demonstrates file handling in Java using character streams by separating even and odd numbers into two different files.

Q4: Briefly explain about the following streams and their usage.

i) FileReader ii) FileWriter

iii) BufferedReader iv)BufferedWriter

Definition

FileReader and FileWriter are classes in Java used for **character stream operations**. They are used to **read and write text data** from and to files.

Explanation

To perform file operations using FileReader and FileWriter:

• Open the file using FileReader for reading
• Open another file using FileWriter for writing
• Read characters one by one using read()
• Write characters using write()
• Close both streams

This method is suitable for **text files**.

Key Points / Properties

• Uses FileReader for reading characters
• Uses FileWriter for writing characters
• Works with **text data only**
• Reads **character by character**
• Must close streams after use

Example with Output

```
import java.io.FileReader;
import java.io.FileWriter;

public class FileCopy {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("source.txt");
            FileWriter fw = new FileWriter("destination.txt");

            int ch;

            while ((ch = fr.read()) != -1) {
```

```
            fw.write(ch);
        }

        fr.close();
        fw.close();

        System.out.println("File copied successfully.");
    } catch (Exception e) {
        System.out.println("Error occurred");
    }
  }
}
```

Output

File copied successfully.

Conclusion

FileReader and FileWriter provide a simple way to handle **text file operations**, reading and writing character data efficiently.

Q4 (continued) BufferedReader and BufferedWriter with example

Definition

BufferedReader and BufferedWriter are classes used for **buffered character streams**. They improve performance by reducing the number of I/O operations.

Explanation

To perform operations using buffered streams:

• Wrap FileReader inside BufferedReader
• Wrap FileWriter inside BufferedWriter
• Read data **line by line** using readLine()
• Write data using write()
• Close both streams

This approach is faster and efficient for large files.

Key Points / Properties

• Uses buffering for **faster performance**
• Reads **line by line**
• Reduces disk access operations
• Suitable for **large text files**
• Must close streams

Example with Output

```
import java.io.*;

public class BufferedCopy {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(new FileReader("source.txt"));
            BufferedWriter bw = new BufferedWriter(new FileWriter("destination.txt"));

            String line;

            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }

            br.close();
            bw.close();

            System.out.println("File copied using buffer.");
        } catch (Exception e) {
            System.out.println("Error occurred");
        }
    }
}
```

Output

File copied using buffer.

Conclusion

BufferedReader and BufferedWriter improve performance by using buffering, making file operations faster and more efficient than basic streams.

Q5: Discuss in detail about various Wrapper Classes in Java with suitable examples

---

Definition

Wrapper classes in Java are classes that **convert primitive data types into objects**. They are part of the java.lang package and allow primitives to be used where objects are required.

Why Wrapper Classes are Needed

- Collections (like ArrayList) store **only objects**, not primitives
- Enable use of primitives in **object-oriented features**
- Allow **null values** (primitives cannot)
- Provide useful **utility methods** (compareTo(), equals(), etc.)

Primitive → Wrapper Mapping

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Types of Conversion

1. Boxing (Manual)

Converting primitive → object manually

Integer obj = Integer.valueOf(10);

Autoboxing

Automatic conversion of primitive → object

```
int a = 100;
Integer obj = a;
```

3. Unboxing

Automatic conversion of object → primitive

```
Integer obj = 200;
int b = obj;
```

Common Methods of Wrapper Classes

Method	Description
parseInt()	Converts String → primitive
valueOf()	Converts primitive/String → object
intValue()	Converts object → primitive
toString()	Converts value → String
compareTo()	Compares two objects
equals()	Checks equality

Example Program

```
public class WrapperDemo {
   public static void main(String[] args) {

      int a = 10;

      // Boxing
      Integer obj = Integer.valueOf(a);

      // Unboxing
      int b = obj;

      // Autoboxing
      Integer x = 20;

      // Auto-unboxing
```

```
        int y = x;

        System.out.println("Wrapper Object: " + obj);
        System.out.println("Primitive Value: " + b);
        System.out.println("Auto-unboxed: " + y);
    }
}
```

Output

Wrapper Object: 10
Primitive Value: 10
Auto-unboxed: 20

Key Points

- Wrapper classes convert **primitive ↔ object**
- Required for **collections and APIs**
- Support **autoboxing and unboxing**
- Provide many **utility methods**
- Present in **java.lang package**

Conclusion

Wrapper classes act as a bridge between primitive data types and objects, enabling Java to support object-oriented programming effectively and making data manipulation more flexible and powerful.

---

6. Write a Java program to illustrate the usage of protected members in a package

Definition

The protected access modifier in Java allows class members (variables and methods) to be accessed **within the same package** and also in **different packages through inheritance (subclasses)**.

Explanation

To illustrate protected members across packages:

- Create a class in one package with protected members
- Create a subclass in another package
- Access the protected members through inheritance
- Direct access using object (non-subclass) is not allowed

Syntax

```
protected dataType variableName;
protected void methodName() {
   // code
}
```

Example Program

File 1: pack1/A.java

```
package pack1;

public class A {
   protected int x = 100;

   protected void show() {
      System.out.println("Protected method");
   }
}
```

File 2: pack2/B.java (Subclass)

```
package pack2;

import pack1.A;

public class B extends A {
   public static void main(String[] args) {

      B obj = new B();

      System.out.println(obj.x); // accessing protected variable
      obj.show();                // accessing protected method
   }
}
```

Output

100
Protected method

Key Points

• protected members are accessible **within same package**
• Accessible in **different package only through inheritance**
• Cannot be accessed using object in non-subclass outside package
• Used to support **controlled inheritance**

Conclusion

The protected access modifier allows controlled access to class members, enabling subclasses to inherit and use them even across different packages.

Q7. Write a Java program to read and display student details stored in a Collection using the Iterator interface.

Definition

A **Collection** in Java is a framework used to store and manage groups of objects.
An **Iterator** is an interface used to **traverse (iterate through) elements of a collection one by one**.

Explanation

To display student details using Iterator:

• Create a Student class
• Store student objects in a collection (e.g., ArrayList)
• Use Iterator to access elements
• Display details using loop

Syntax

```
Iterator<Type> it = collection.iterator();

while(it.hasNext()) {
   Type obj = it.next();
}
```

Program

```
import java.util.*;

class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println(id + " " + name);
    }
}

public class Main {
    public static void main(String[] args) {

        ArrayList<Student> list = new ArrayList<>();

        list.add(new Student(101, "Ravi"));
        list.add(new Student(102, "Anu"));
        list.add(new Student(103, "Kiran"));

        Iterator<Student> it = list.iterator();

        while (it.hasNext()) {
            Student s = it.next();
            s.display();
        }
    }
}
```

Output

```
101 Ravi
102 Anu
103 Kiran
```

Key Points

• ArrayList stores objects dynamically
• Iterator is used for traversal
• hasNext() checks availability
• next() returns next element
• Works for all collection types

Conclusion

The Iterator interface provides a simple and efficient way to traverse elements in a collection and display stored data.

Q8: Explain ArrayList and LinkedList. Compare their features, performance, and applications.

Definition

ArrayList and LinkedList are classes in the **Java Collections Framework** that implement the **List interface**.

• **ArrayList** → a dynamic array that can grow or shrink in size
• **LinkedList** → a doubly linked list where elements are connected using nodes

Explanation

An **ArrayList** stores elements in **contiguous memory locations**, similar to arrays. It allows fast access using index but is slower for insertion or deletion in the middle due to shifting of elements.

A **LinkedList** stores elements as **nodes**, where each node contains data and references to the next and previous nodes. It is efficient for insertion and deletion but slower for accessing elements.

Key Points / Properties

ArrayList

• Uses dynamic array internally
• Provides fast random access (**O(1)**)
• Slow insertion/deletion in middle (**O(n)**)
• Consumes less memory

LinkedList

• Uses doubly linked list
• Slow access (**O(n)**)

• Fast insertion/deletion (**O(1)**)
• Requires extra memory for pointers

Comparison Table

Feature	**ArrayList**	**LinkedList**
Data Structure	Dynamic Array	Doubly Linked List
Access Time	Fast (O(1))	Slow (O(n))
Insertion/Deletion	Slow (O(n))	Fast (O(1))
Memory Usage	Less	More (extra pointers)
Traversal	Easy using index	Sequential traversal
Performance	Better for searching	Better for insert/delete
Implementation	Resizable array	Node-based structure

Example with Output

```
import java.util.ArrayList;
import java.util.LinkedList;

public class ListDemo {
   public static void main(String[] args) {

      // ArrayList
      ArrayList<String> arrList = new ArrayList<>();
      arrList.add("Apple");
      arrList.add("Banana");

      // LinkedList
      LinkedList<String> linkList = new LinkedList<>();
      linkList.add("Car");
      linkList.add("Bike");

      System.out.println("ArrayList: " + arrList);
      System.out.println("LinkedList: " + linkList);
   }
}
```

Output

ArrayList: [Apple, Banana]
LinkedList: [Car, Bike]

Applications

ArrayList

• When frequent **data access/searching** is required
• Used in **read-heavy applications**

LinkedList

• When frequent **insertions and deletions** are required
• Used in **queue, stack, and dynamic memory applications**

Conclusion

ArrayList and LinkedList serve different purposes. ArrayList is efficient for fast access and searching, while LinkedList is suitable for frequent modifications. The choice depends on application requirements.

9. Explain Comparable and Comparator interfaces with suitable examples

Definition

Comparable Interface

Comparable is an interface in Java used to define the **natural ordering of objects**. It is present in java.lang package and uses the compareTo() method.

Comparator Interface

Comparator is an interface used to define **custom sorting**. It is present in java.util package and uses the compare() method.

Syntax

```
class ClassName implements Comparable<ClassName> {
    public int compareTo(ClassName obj) {
        // logic
    }
}
```

```
class ClassName implements Comparator<ClassName> {
   public int compare(ClassName o1, ClassName o2) {
      // logic
   }
}
```

Explanation

Comparable

- Used for **default (natural) sorting**
- Sorting logic is written **inside the class**
- Uses compareTo() method

Comparator

- Used for **custom sorting**
- Sorting logic is written **outside the class**
- Uses compare() method

Example (Comparable)

```
import java.util.*;

class Student implements Comparable<Student> {
   int id;
   String name;

   Student(int id, String name) {
      this.id = id;
      this.name = name;
   }

   public int compareTo(Student s) {
      return this.id - s.id;
   }
}
public class Main {
   public static void main(String[] args) {
```

```
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(3, "Ravi"));
        list.add(new Student(1, "Anu"));
        list.add(new Student(2, "Kiran"));

        Collections.sort(list);

        for (Student s : list) {
            System.out.println(s.id + " " + s.name);
        }
    }
}
```

Output

```
1 Anu
2 Kiran
3 Ravi
```

Example (Comparator)

```
import java.util.*;

class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

class NameComparator implements Comparator<Student> {
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name);
    }
}

public class Main {
    public static void main(String[] args) {

        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student(3, "Ravi"));
        list.add(new Student(1, "Anu"));
        list.add(new Student(2, "Kiran"));
```

```
        Collections.sort(list, new NameComparator());

        for (Student s : list) {
            System.out.println(s.id + " " + s.name);
        }
    }
}
```

Output

1 Anu
2 Kiran
3 Ravi

Comparison Table

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo()	compare()
Sorting Type	Natural order	Custom order
Location of Logic	Inside class	Outside class
Flexibility	Less	More
Multiple Sorting	Not possible	Possible

Key Points

• Comparable → default sorting
• Comparator → custom sorting
• Used in **Collections.sort()**
• Helps in object comparison

Conclusion

Comparable and Comparator interfaces are used for sorting objects in Java. Comparable provides natural ordering, while Comparator allows flexible and multiple sorting strategies.

10. Explain about HashSet and TreeSet. How do they store elements and maintain uniqueness?

Definition

HashSet

HashSet is a class in the Java Collections Framework that implements the Set interface. It stores **unique elements** using a **hashing mechanism** and does **not maintain any order**.

TreeSet

TreeSet is a class that the Set interface using a **tree structure (Red-Black Tree)**. It stores elements in **sorted (ascending) order** and does not allow duplicates implements.

Syntax

```
HashSet<Type> set1 = new HashSet<>();

TreeSet<Type> set2 = new TreeSet<>();
```

How They Store Elements

HashSet

- Uses **Hash Table (HashMap internally)**
- Stores elements based on **hash codes**
- Elements are placed into **buckets**
- Order is **not maintained**

TreeSet

- Uses **Red-Black Tree (balanced binary search tree)**
- Elements are stored in **sorted order**
- Uses **compareTo() / Comparator** for ordering

How They Maintain Uniqueness

HashSet

- Uses **hashCode()** to find bucket

- Uses **equals()** to check duplicates
- Duplicate elements are **not added**

TreeSet

- Uses **compareTo() / compare()**
- If comparison returns **0 → duplicate**
- Duplicate elements are **not stored**

Example Program

```
import java.util.*;

public class SetDemo {
   public static void main(String[] args) {

      HashSet<Integer> hs = new HashSet<>();
      hs.add(10);
      hs.add(20);
      hs.add(10); // duplicate

      TreeSet<Integer> ts = new TreeSet<>();
      ts.add(30);
      ts.add(10);
      ts.add(20);
      ts.add(10); // duplicate

      System.out.println("HashSet: " + hs);
      System.out.println("TreeSet: " + ts);
   }
}
```

Output

```
HashSet: [10, 20]
TreeSet: [10, 20, 30]
```

Comparison Table

Feature	HashSet	TreeSet
Structure	Hash Table	Red-Black Tree
Order	No order	Sorted order
Performance	Faster (O(1))	Slower (O(log n))
Duplicate Handling	hashCode + equals	compareTo / compare
Null Values	Allows one null	Does not allow null
Use Case	Fast operations	Sorted data

Key Points

• Both implement Set interface
• Do not allow duplicate elements
• HashSet → fast but unordered
• TreeSet → sorted but slower
• Used for storing unique elements

Conclusion

HashSet and TreeSet both store unique elements, but differ in structure and ordering. HashSet is faster, while TreeSet maintains sorted order using a tree structure.